Traffic Light Control

in Assembly Language

A complete beginner guide for Ubuntu Terminal (x86-64 / NASM)
Shareable student document - theory, steps, full source code, and exercises

---

1. Introduction

This project shows how to design a simple traffic light controller using assembly language on Ubuntu. Instead of controlling real hardware lamps, the program simulates the lights in the terminal: it prints colored text for RED, GREEN, and YELLOW, waits for a set time, then repeats.

This matches how a real traffic light controller works: turn on one lamp, wait, switch to the next lamp, wait, and loop forever.

2. Learning objectives

- Understand a repeating control loop (state machine idea)
- Use Linux system calls from assembly (write and nanosleep)
- Store messages and timing values in the .data section
- Use registers (rax, rdi, rsi, rdx) for system call arguments
- Build and run an assembly program with nasm and ld

3. How the traffic light maps to code

Physical world This program

```
Red lamp ON                    Print red text, then sleep
Green lamp ON                  Print green text, then sleep
Yellow lamp ON                 Print yellow text, then sleep
Controller repeats             jmp .loop
```

Default timing used in this project:

- RED - 3 seconds (STOP)
- GREEN - 3 seconds (GO)
- YELLOW - 1 second (CAUTION)

You can change these values in the .data section of traffic_light.asm.

4. Requirements (Ubuntu)

Install the assembler and linker tools:

```
sudo apt update
sudo apt install nasm binutils
```

- nasm - assembler (turns .asm into an object file .o)
- ld - linker (turns the .o file into an executable program)

5. How to build and run

Option A - using make

```
make
./traffic_light
```

Option B - manual commands

```
nasm -f elf64 traffic_light.asm -o traffic_light.o
ld traffic_light.o -o traffic_light
./traffic_light
```

Press Ctrl+C to stop the program.

To remove build files later:

```
make clean
```

6. Step-by-step: what happens when it runs

- 1. _start begins (Linux entry point for assembly programs).
- 2. The screen is cleared using an ANSI escape sequence.
- 3. The title and RED message are printed with sys_write.
- 4. The program sleeps 3 seconds with sys_nanosleep.
- 5. The same process runs for GREEN, then YELLOW.
- 6. jmp .loop jumps back to the beginning and repeats forever.

7. Important code ideas explained

7.1 Printing to the terminal (sys_write)

```
mov rax, 1      ; syscall number for write
mov rdi, 1      ; file descriptor 1 = stdout (terminal)
mov rsi, ...    ; address of the text to print
mov rdx, ...    ; how many bytes to print
syscall
```

7.2 Waiting / delay (sys_nanosleep)

```
; delay_red: dq 3, 0   means 3 seconds, 0 nanoseconds
mov rax, 35     ; syscall number for nanosleep
mov rdi, delay_red
xor rsi, rsi
syscall
```

7.3 Terminal colors (ANSI escape codes)

ANSI codes change text color in the terminal. In the .asm file, the number 27 is the ESC character.

- ESC[1;31m - bold red
- ESC[1;32m - bold green
- ESC[1;33m - bold yellow
- ESC[0m - reset color

7.4 Program structure

- .data section - stores messages, colors, and delay times
- .text section - contains the instructions the CPU runs
- print / clear_screen / sleep - small helper routines
- .loop + jmp - keeps the traffic light cycling

8. Project files included

- traffic_light.asm - main assembly source code
- Makefile - shortcuts to build, run, and clean
- README.md - short project readme
- Traffic_Light_Assembly_Guide.pdf - this document

9. Full Makefile

```
# Traffic light control - build helpers

ASM = nasm
LD  = ld

TARGET = traffic_light
SRC    = traffic_light.asm
OBJ    = traffic_light.o

.PHONY: all clean run

all: $(TARGET)
```

```
$(TARGET): $(OBJ)
    $(LD) $(OBJ) -o $(TARGET)

$(OBJ): $(SRC)
    $(ASM) -f elf64 $(SRC) -o $(OBJ)

run: $(TARGET)
    ./$(TARGET)

clean:
    rm -f $(OBJ) $(TARGET)
```

10. Full assembly source code (traffic_light.asm)

Copy this file as traffic_light.asm, then build and run using the commands in Section 5.

```
; ==========================================================================
; traffic_light.asm  -  Simple traffic light controller (x86-64 Linux / NASM)
; ==========================================================================
; WHAT THIS PROGRAM DOES
;   Simulates a traffic light in the Ubuntu terminal:
;     RED    (stop)     -> wait
;     GREEN  (go)       -> wait
;     YELLOW (caution)  -> wait
;     then repeats
;
; HOW IT WORKS (big picture)
;   1. Print a colored light using ANSI escape codes
;   2. Sleep for a few seconds (Linux nanosleep system call)
;   3. Jump back and show the next light
;
; BUILD & RUN
;   nasm -f elf64 traffic_light.asm -o traffic_light.o
;   ld traffic_light.o -o traffic_light
;   ./traffic_light
;   Press Ctrl+C to stop
; ==========================================================================

section .data

    ; Clear screen + move cursor home (ANSI)
    clear_seq:      db 27, "[2J", 27, "[H"
    clear_len:      equ $ - clear_seq

    ; Title line
    title:          db "=== TRAFFIC LIGHT CONTROL ===", 10, 10
    title_len:      equ $ - title

    ; RED light: bright red circle + label
    ; 27 = ESC character for ANSI color codes
    red_msg:        db 27, "[1;31m"          ; bold red text
                    db "    ( R )", 10
                    db "     RED", 10
                    db "    STOP", 10
                    db 27, "[0m"             ; reset color
    red_len:        equ $ - red_msg

    ; GREEN light
    green_msg:      db 27, "[1;32m"          ; bold green text
                    db "    ( G )", 10
                    db "    GREEN", 10
                    db "     GO", 10
                    db 27, "[0m"
    green_len:      equ $ - green_msg

    ; YELLOW light
    yellow_msg:     db 27, "[1;33m"          ; bold yellow text
                    db "    ( Y )", 10
                    db "   YELLOW", 10
                    db "  CAUTION", 10
                    db 27, "[0m"
```

```
    yellow_len:     equ $ - yellow_msg

    hint:           db 10, "Press Ctrl+C to exit", 10
    hint_len:       equ $ - hint
```

```
    ; timespec for nanosleep: { tv_sec, tv_nsec }
    ; Change tv_sec to make lights stay longer/shorter
    delay_red:      dq 3, 0                 ; RED stays 3 seconds
    delay_green:    dq 3, 0                 ; GREEN stays 3 seconds
    delay_yellow:   dq 1, 0                 ; YELLOW stays 1 second

section .text
    global _start

_start:
.loop:
    ; ----- RED -----
    call clear_screen
    mov rsi, title
    mov rdx, title_len
    call print
    mov rsi, red_msg
    mov rdx, red_len
    call print
    mov rsi, hint
    mov rdx, hint_len
    call print
    mov rdi, delay_red
    call sleep

    ; ----- GREEN -----
    call clear_screen
    mov rsi, title
    mov rdx, title_len
    call print
    mov rsi, green_msg
    mov rdx, green_len
    call print
    mov rsi, hint
    mov rdx, hint_len
    call print
    mov rdi, delay_green
    call sleep

    ; ----- YELLOW -----
    call clear_screen
    mov rsi, title
    mov rdx, title_len
    call print
    mov rsi, yellow_msg
    mov rdx, yellow_len
    call print
    mov rsi, hint
    mov rdx, hint_len
    call print
```

```
    mov rdi, delay_yellow
    call sleep

    jmp .loop                   ; repeat forever

; ---------------------------------------------------------------------------
; print: write RSI (buffer) of length RDX to stdout
;   uses Linux syscall write (rax = 1)
; ---------------------------------------------------------------------------
print:
```

```
    mov rax, 1                  ; sys_write
```

```
    mov rdi, 1                  ; fd = 1 (stdout)
    syscall
    ret

; ------------------------------------------------------------------------------
; clear_screen: wipe terminal and home cursor
; ------------------------------------------------------------------------------
clear_screen:
    mov rsi, clear_seq
    mov rdx, clear_len
    call print
    ret

; ------------------------------------------------------------------------------
; sleep: nanosleep using pointer in RDI (struct timespec *)
;   syscall nanosleep (rax = 35)
; ------------------------------------------------------------------------------
sleep:
    mov rax, 35                 ; sys_nanosleep
    xor rsi, rsi                ; remaining = NULL
    syscall
    ret
```

11. Practice exercises

- 1. Change RED to stay 5 seconds by editing delay_red in the .data section.
- 2. Change the order to Red -> Yellow -> Green (used in some countries).
- 3. Add a blinking yellow state (print, short sleep, clear, short sleep).
- 4. Design a two-way intersection: North-South green while East-West is red, then switch.

12. Quick troubleshooting

- Command 'nasm' not found -> run: sudo apt install nasm
- Permission denied on ./traffic_light -> run: chmod +x traffic_light
- No colors shown -> use a normal terminal (GNOME Terminal, xterm, etc.)
- Program does not stop -> press Ctrl+C

13. Summary

A traffic light controller is a loop of states with timing. In assembly on Ubuntu, we implement that loop using print (sys_write), delay (sys_nanosleep), and jump (jmp). This document contains the full explanation and complete source so another person can follow, build, and run the project on Ubuntu.